Design and Modeling of Complex Software Architectures

Lab Assignment 04, Tactical Design & Event Storming

Application Scenario 4: Instant Mobility Platform

Team: The Winx | Summer Semester 2026

Task 1: Tactical Design

Each bounded context from Assignment 03 has been assigned to one team member. That member is responsible for the tactical DDD design and the UML class diagram of their context, and will later own its implementation. The remaining contexts will be mocked.

1.1 Team Assignment

BC	Bounded Context	Responsible Member	Implementation
BC-01	**Identity & Access**	Team Member A	Full implementation
BC-02	**Fleet Management**	Team Member B	Full implementation
BC-03	**Booking**	Team Member C	Full implementation
BC-04	**Payment**	Team Member D	Full implementation
BC-05	**Rating**	Team Member E	Full implementation (remaining contexts mocked)

1.2 DDD Building Block Reference

The following UML stereotypes are used in all class diagrams below, following the notation from Paper 01 (Table 1) and Paper 02 (Table 3) provided in the lab materials.

Stereotype	Fill	Border	Role in design
<<AggregateRoot>>	Amber	Thick solid	Entry point to an aggregate. All access to internal objects goes through it. Enforces aggregate invariants.
<<Entity>>	Light yellow	Solid	Has a unique identity. State changes over time. Tracked by ID rather than attribute values.
<<ValueObject>>	Green	Dashed	No identity. Defined entirely by its attributes. Immutable, replaced, never mutated.

<<Service>>	Blue	Solid	Stateless. Contains domain logic that does not naturally belong to an entity or value object.
<<Repository>>	Purple	Solid	Provides persistence operations for aggregate roots. Hides storage implementation details.
<<DomainEvent>>	Orange	Solid	Records that something significant happened in the domain. Immutable. Published after state change.

1.3 Tactical Design per Bounded Context

BC-01 Identity & Access , Team Member A

This context owns all authentication and account management logic. The two aggregate roots (UserAccount and ProviderAccount) encapsulate credential validation and registration. Value Objects enforce immutability of credentials and contact data. No booking or fleet logic exists here.

Aggregates

Aggregate Root	Members	Key Invariants
UserAccount	PersonalInfo, Email, Password, PhoneNumber	Email must be unique. Password is always stored hashed. Account cannot be deleted while active bookings exist.
ProviderAccount	CompanyInfo, Email, Password, PhoneNumber	Email must be unique. Password always hashed. Cannot be deleted while vehicles exist.

Entities

Entity	Identity Attribute	Reason, why not a Value Object?
UserAccount	`useraccountId : Long`	Has persistent identity (userId). Lifecycle: PENDING → ACTIVE → DEACTIVATED.
ProviderAccount	`provideraccountId : Long`	Has persistent identity (providerId). Lifecycle: PENDING → ACTIVE → DEACTIVATED.

Value Objects

Value Object	Attributes	Why immutable / no identity?
Email	`value:String`	Immutable after set. Enforces format validation on construction.
Password	`hash:String`	Immutable. Never stored in plain text. Provides matches() comparison method.
PhoneNumber	`value:String`	Immutable. Validated format. Shared between PersonalInfo and CompanyInfo.

PersonalInfo	`name, dateOfBirth, phoneNumber`	Groups user personal data. Immutable snapshot, replaced, not mutated.
CompanyInfo	`companyName, contactName, phoneNumber`	Groups provider business data. Immutable snapshot.
AuthToken	`value:String, expiresAt:DateTime`	Immutable session token returned on successful authentication.

Domain Services

Service	Key Operations
AuthenticationService	`authenticate(email,password):AuthToken, validateToken(token):Principal`
RegistrationService	`registerUser(email,password,info):UserAccount, registerProvider(email,password,info):ProviderAccount`

Repositories

Repository	Methods
UserRepository	`findById(id), findByEmail(email), save(user), delete(id)`
ProviderRepository	`findById(id), findByEmail(email), save(provider), delete(id)`

Domain Events

Domain Event	Trigger Condition
UserRegistered	RegistrationService successfully creates a UserAccount
ProviderRegistered	RegistrationService successfully creates a ProviderAccount
UserAuthenticated	AuthenticationService validates credentials and issues a token

PlantUML Source:

```
@startuml BC-01: Identity & Access
skinparam classBackgroundColor<<AggregateRoot>> #FFF3CD
skinparam classBackgroundColor<<ValueObject>> #D4EDDA
skinparam classBackgroundColor<<Service>> #CCE5FF
skinparam classBackgroundColor<<Repository>> #E2D9F3
skinparam classBackgroundColor<<DomainEvent>> #FFE0B2
skinparam classBorderColor<<AggregateRoot>> #856404
skinparam classBorderThickness<<AggregateRoot>> 2
skinparam classBorderStyle<<ValueObject>> dashed

class UserAccount <<AggregateRoot>> {
  + userId : Long
  + registeredAt : DateTime
  + status : AccountStatus
  --
  + authenticate(pwd) : AuthToken
  + deactivate() : void
```

```
}
class ProviderAccount <<AggregateRoot>> {
  + providerId : Long
  + registeredAt : DateTime
  + status : AccountStatus
  --
  + authenticate(pwd) : AuthToken
}
class PersonalInfo <<ValueObject>> {
  + name : String
  + dateOfBirth : Date
}
class CompanyInfo <<ValueObject>> {
  + companyName : String
  + contactName : String
}
class Email <<ValueObject>> {
  + value : String
  --
  + validate() : boolean
}
class Password <<ValueObject>> {
  + hash : String
  --
  + matches(plain) : boolean
}
class PhoneNumber <<ValueObject>> { + value : String }
class AuthToken <<ValueObject>> {
  + value : String
  + expiresAt : DateTime
}
class AuthenticationService <<Service>> {
  + authenticate(email,pwd) : AuthToken
  + validateToken(token) : Principal
}
class RegistrationService <<Service>> {
  + registerUser(...) : UserAccount
  + registerProvider(...) : ProviderAccount
}
interface UserRepository <<Repository>> {
  + findById(id) : UserAccount
  + findByEmail(e) : UserAccount
  + save(u) : void
}
interface ProviderRepository <<Repository>> {
  + findById(id) : ProviderAccount
  + findByEmail(e) : ProviderAccount
  + save(p) : void
}
class UserRegistered <<DomainEvent>> { + userId:Long + email:String }
class ProviderRegistered <<DomainEvent>> { + providerId:Long + email:String }

UserAccount *-- PersonalInfo
UserAccount *-- Email
UserAccount *-- Password
PersonalInfo *-- PhoneNumber
ProviderAccount *-- CompanyInfo
ProviderAccount *-- Email
ProviderAccount *-- Password
CompanyInfo *-- PhoneNumber
AuthenticationService ..> UserRepository
AuthenticationService ..> ProviderRepository
```

```
RegistrationService ..> UserRepository
RegistrationService ..> ProviderRepository
RegistrationService ..> UserRegistered : publishes
RegistrationService ..> ProviderRegistered : publishes
@enduml
```

BC-02 Fleet Management , Team Member B

This context is the single source of truth for vehicles. The Vehicle aggregate encapsulates location, pricing, and restriction data as embedded value objects so that vehicle integrity can be enforced as a unit. Provider identity comes from BC-01 via ACL, only the providerId reference is stored here.

Aggregates

Aggregate Root	Members	Key Invariants
Vehicle	VehicleLocation, PricingPolicy, UsageRestrictions	pricePerUnit must be > 0. A Vehicle cannot be deleted while its status is BOOKED. Restrictions are optional but validated as non-negative when present.

Entities

Entity	Identity Attribute	Reason, why not a Value Object?
Vehicle	`vehicleId : Long`	Has persistent identity (vehicleId). Status, location, and pricing change over its lifetime.

Value Objects

Value Object	Attributes	Why immutable / no identity?
VehicleLocation	`latitude:Double, longitude:Double`	Immutable GPS snapshot. Replaced entirely on each location update.
PricingPolicy	`pricePerUnit:BigDecimal, billingModel:BillingModel`	Immutable. Changing the price replaces the entire value object.
UsageRestrictions	`maxDurationMins, maxKm, minAge, maxPersons (all nullable Integer)`	Immutable. Groups all optional restriction rules. Null means no restriction.
VehicleType	`E_SCOOTER | BICYCLE | E_BIKE | E_CAR`	Enum-backed value object. Fixed vocabulary, cannot be extended at runtime.

Domain Services

Service	Key Operations
VehicleRegistrationService	`createVehicle(providerId, type, desc, pricing, restrictions):Vehicle`
FleetStatusService	`getFleetOverview(providerId):List<VehicleStatus>, updateLocation(vehicleId, location):void`

VehicleAvailabilityService	`findAvailableNear(lat, lon, radiusKm):List<Vehicle>, findById(id):Vehicle`

Repositories

Repository	Methods
VehicleRepository	`findById(id), findByProviderId(pid), findAvailableNear(lat,lon,radius), save(v), delete(id)`

Domain Events

Domain Event	Trigger Condition
VehicleCreated	VehicleRegistrationService successfully registers a new vehicle
VehicleStatusUpdated	Booking context sends status-change command (AVAILABLE ↔ BOOKED)
VehicleLocationUpdated	FleetStatusService records new GPS coordinates
VehicleDeleted	Provider deletes a vehicle that is currently AVAILABLE

PlantUML Source:

```
@startuml BC-02: Fleet Management
skinparam classBackgroundColor<<AggregateRoot>> #FFF3CD
skinparam classBackgroundColor<<ValueObject>> #D4EDDA
skinparam classBackgroundColor<<Service>> #CCE5FF
skinparam classBackgroundColor<<Repository>> #E2D9F3
skinparam classBackgroundColor<<DomainEvent>> #FFE0B2
skinparam classBorderThickness<<AggregateRoot>> 2
skinparam classBorderStyle<<ValueObject>> dashed

class Vehicle <<AggregateRoot>> {
  + vehicleId : Long
  + providerId : Long
  + description : String
  + status : VehicleStatus
  --
  + markBooked() : void
  + markAvailable() : void
  + updateLocation(loc) : void
}
class VehicleLocation <<ValueObject>> {
  + latitude : Double
  + longitude : Double
}
class PricingPolicy <<ValueObject>> {
  + pricePerUnit : BigDecimal
  + billingModel : BillingModel
}
class UsageRestrictions <<ValueObject>> {
  + maxDurationMins : Integer
  + maxKilometers : Integer
  + minAge : Integer
  + maxPersons : Integer
}
enum VehicleType { E_SCOOTER / BICYCLE / E_BIKE / E_CAR }
```

```
enum VehicleStatus { AVAILABLE / BOOKED }
enum BillingModel { PER_HOUR / PER_KILOMETER }
class VehicleRegistrationService <<Service>> {
  + createVehicle(...) : Vehicle
}
class FleetStatusService <<Service>> {
  + getFleetOverview(pid) : List
  + updateLocation(vid, loc) : void
}
interface VehicleRepository <<Repository>> {
  + findById(id) : Vehicle
  + findByProviderId(pid) : List
  + findAvailableNear(lat,lon,r) : List
  + save(v) : void
  + delete(id) : void
}
class VehicleCreated <<DomainEvent>> { + vehicleId:Long + providerId:Long }
class VehicleStatusUpdated <<DomainEvent>> { + vehicleId:Long + newStatus:VehicleStatus }

Vehicle *-- VehicleLocation
Vehicle *-- PricingPolicy
Vehicle *-- UsageRestrictions
Vehicle --> VehicleType
Vehicle --> VehicleStatus
PricingPolicy --> BillingModel
VehicleRepository ..> Vehicle
VehicleRegistrationService ..> VehicleRepository
FleetStatusService ..> VehicleRepository
VehicleRegistrationService ..> VehicleCreated : publishes
FleetStatusService ..> VehicleStatusUpdated : publishes
@enduml
```

BC-03 Booking , Team Member C

The Booking context is the operational heart of the platform. The Booking aggregate root enforces all ride lifecycle invariants. A VehicleSnapshot value object captures pricing at booking-time to prevent retroactive pricing changes from affecting in-progress rides. Cost computation and payment triggering are handled by domain services.

Aggregates

Aggregate Root	Members	Key Invariants
Booking	RideLocation (start/end), TimeInterval, VehicleSnapshot, RideSummary	endTime must be after startTime. A COMPLETED booking cannot transition to CANCELLED. totalCost must be computed before status reaches COMPLETED. Only one active booking per User at a time.

Entities

Entity	Identity Attribute	Reason, why not a Value Object?

Booking	`bookingId : Long`	Has persistent identity (bookingId). Status transitions define its entire lifecycle: ACTIVE → COMPLETED or CANCELLED.

Value Objects

Value Object	Attributes	Why immutable / no identity?
RideLocation	`latitude:Double, longitude:Double`	Immutable GPS snapshot captured at start and end of ride.
TimeInterval	`startTime:DateTime, endTime:DateTime`	Immutable. endTime is null during ACTIVE ride. Replaced when ride ends.
VehicleSnapshot	`vehicleId, type, pricePerUnit, billingModel`	Immutable snapshot of pricing at booking time. Decouples Booking from future vehicle price changes.
RideSummary	`distanceKm:Double, totalCost:BigDecimal`	Immutable result of cost computation. Null until ride is COMPLETED.

Domain Services

Service	Key Operations
VehicleSearchService	`findAvailableNear(userId, lat, lon, filters):List<VehicleSnapshot>`
BookingService	`createBooking(userId, vehicleId):Booking, endBooking(bookingId, endLat, endLon):Booking, cancelBooking(bookingId):Booking`
CostCalculationService	`computeCost(snapshot:VehicleSnapshot, interval:TimeInterval, distKm:Double):BigDecimal`
RestrictionValidator	`validate(userId:UserRef, vehicle:VehicleSnapshot):void, throws if age/time/km restrictions violated`

Repositories

Repository	Methods
BookingRepository	`findById(id), findByUserId(uid), findActiveByUserId(uid), findByVehicleId(vid), save(b)`

Domain Events

Domain Event	Trigger Condition
BookingCreated	BookingService creates a new ACTIVE booking; vehicle status updated to BOOKED
BookingCompleted	BookingService ends ride, cost computed, payment trigger sent
BookingCancelled	BookingService cancels an ACTIVE booking; vehicle released
PaymentTriggered	Published after BookingCompleted; consumed by Payment context

PlantUML Source:

```
@startuml BC-03: Booking
skinparam classBackgroundColor<<AggregateRoot>> #FFF3CD
skinparam classBackgroundColor<<ValueObject>> #D4EDDA
skinparam classBackgroundColor<<Service>> #CCE5FF
skinparam classBackgroundColor<<Repository>> #E2D9F3
skinparam classBackgroundColor<<DomainEvent>> #FFE0B2
skinparam classBorderThickness<<AggregateRoot>> 2
skinparam classBorderStyle<<ValueObject>> dashed

class Booking <<AggregateRoot>> {
  + bookingId : Long
  + userId : Long
  + status : BookingStatus
  --
  + complete(endLoc, distKm, cost) : void
  + cancel() : void
}
class RideLocation <<ValueObject>> {
  + latitude : Double
  + longitude : Double
}
class TimeInterval <<ValueObject>> {
  + startTime : DateTime
  + endTime : DateTime
}
class VehicleSnapshot <<ValueObject>> {
  + vehicleId : Long
  + type : VehicleType
  + pricePerUnit : BigDecimal
  + billingModel : BillingModel
}
class RideSummary <<ValueObject>> {
  + distanceKm : Double
  + totalCost : BigDecimal
}
enum BookingStatus { ACTIVE / COMPLETED / CANCELLED }
class BookingService <<Service>> {
  + createBooking(uid, vid) : Booking
  + endBooking(bid, loc) : Booking
  + cancelBooking(bid) : Booking
}
class CostCalculationService <<Service>> {
  + computeCost(snap, interval, km) : BigDecimal
}
class VehicleSearchService <<Service>> {
  + findAvailableNear(lat,lon,filters) : List
}
class RestrictionValidator <<Service>> {
  + validate(userId, snapshot) : void
}
interface BookingRepository <<Repository>> {
  + findById(id) : Booking
  + findByUserId(uid) : List
  + findActiveByUserId(uid) : Booking
  + save(b) : void
}
class BookingCreated <<DomainEvent>> { + bookingId:Long + userId:Long + vehicleId:Long }
class BookingCompleted <<DomainEvent>> { + bookingId:Long + totalCost:BigDecimal }
class BookingCancelled <<DomainEvent>> { + bookingId:Long + vehicleId:Long }
```

```
class PaymentTriggered <<DomainEvent>> { + bookingId:Long + amount:BigDecimal +
userId:Long }

Booking *-- RideLocation
Booking *-- TimeInterval
Booking *-- VehicleSnapshot
Booking *-- RideSummary
Booking --> BookingStatus
BookingService ..> BookingRepository
BookingService ..> CostCalculationService
BookingService ..> RestrictionValidator
BookingService ..> BookingCreated : publishes
BookingService ..> BookingCompleted : publishes
BookingService ..> BookingCancelled : publishes
BookingService ..> PaymentTriggered : publishes
@enduml
```

BC-04 Payment , Team Member D

The Payment context is triggered by the PaymentTriggered domain event from the Booking context. It is a leaf context with no outward dependencies. The Payment aggregate enforces financial consistency: amount must match the booking total, and status transitions are strictly controlled (PENDING → PAID or FAILED only).

Aggregates

Aggregate Root	Members	Key Invariants
Payment	Money, PaymentMethod, PaymentResult	Amount must equal the bookingId's totalCost received from the event. Status can only transition PENDING → PAID or PENDING → FAILED, never backwards. A PAID payment cannot be modified.

Entities

Entity	Identity Attribute	Reason, why not a Value Object?
Payment	`paymentId : Long`	Has persistent identity (paymentId). Status transitions define its lifecycle.

Value Objects

Value Object	Attributes	Why immutable / no identity?
Money	`amount:BigDecimal, currency:String`	Immutable monetary amount. Enforces non-negative values. Currency code validated (ISO 4217).
PaymentMethod	`type:String, maskedReference:String`	Immutable reference to the payment instrument used. Stored masked for PCI compliance.
PaymentResult	`status:PaymentStatus, paidAt:DateTime, failureReason:String`	Immutable outcome of payment processing. Null paidAt when FAILED or PENDING.

Domain Services

Service	Key Operations
PaymentProcessingService	`initiatePayment(bookingId, amount, userId):Payment, retryPayment(paymentId):Payment`
PaymentGatewayAdapter	`charge(money, method):PaymentResult , infrastructure service; adapts to external payment provider API`

Repositories

Repository	Methods
PaymentRepository	`findById(id), findByBookingId(bid), save(p)`

Domain Events

Domain Event	Trigger Condition
PaymentInitiated	PaymentProcessingService creates a new PENDING Payment in response to PaymentTriggered event
PaymentSucceeded	PaymentGatewayAdapter returns successful charge; status set to PAID
PaymentFailed	PaymentGatewayAdapter returns failure; status set to FAILED with reason

PlantUML Source:

```
@startuml BC-04: Payment
skinparam classBackgroundColor<<AggregateRoot>> #FFF3CD
skinparam classBackgroundColor<<ValueObject>> #D4EDDA
skinparam classBackgroundColor<<Service>> #CCE5FF
skinparam classBackgroundColor<<Repository>> #E2D9F3
skinparam classBackgroundColor<<DomainEvent>> #FFE0B2
skinparam classBorderThickness<<AggregateRoot>> 2
skinparam classBorderStyle<<ValueObject>> dashed

class Payment <<AggregateRoot>> {
  + paymentId : Long
  + bookingId : Long
  + userId : Long
  --
  + markPaid(paidAt) : void
  + markFailed(reason) : void
}
class Money <<ValueObject>> {
  + amount : BigDecimal
  + currency : String
}
class PaymentMethod <<ValueObject>> {
  + type : String
  + maskedReference : String
}
class PaymentResult <<ValueObject>> {
  + status : PaymentStatus
  + paidAt : DateTime
```

```
    + failureReason : String
}
enum PaymentStatus { PENDING / PAID / FAILED }
class PaymentProcessingService <<Service>> {
  + initiatePayment(bookingId,amount,uid) : Payment
  + retryPayment(paymentId) : Payment
}
class PaymentGatewayAdapter <<Service>> {
  + charge(money, method) : PaymentResult
}
interface PaymentRepository <<Repository>> {
  + findById(id) : Payment
  + findByBookingId(bid) : Payment
  + save(p) : void
}
class PaymentInitiated <<DomainEvent>> { + paymentId:Long + bookingId:Long +
amount:BigDecimal }
class PaymentSucceeded <<DomainEvent>> { + paymentId:Long + paidAt:DateTime }
class PaymentFailed <<DomainEvent>> { + paymentId:Long + reason:String }

Payment *-- Money
Payment *-- PaymentMethod
Payment *-- PaymentResult
PaymentResult --> PaymentStatus
PaymentProcessingService ..> PaymentRepository
PaymentProcessingService ..> PaymentGatewayAdapter
PaymentProcessingService ..> PaymentInitiated : publishes
PaymentProcessingService ..> PaymentSucceeded : publishes
PaymentProcessingService ..> PaymentFailed : publishes
@enduml
```

BC-05 Rating , Team Member E

The Rating context allows users to submit feedback after completing a ride. The aggregate enforces the one-rating-per-booking invariant and validates score ranges. Vehicle and Provider data arrive as lightweight snapshots from Fleet Management via ACL. No full fleet entities exist in this context.

Aggregates

Aggregate Root	Members	Key Invariants
Rating	Review, RatingTarget	vehicleScore and providerScore must be 1–5. Exactly one Rating may exist per bookingId. Rating can only be submitted if the referenced booking is COMPLETED. Once submitted, a Rating is immutable.

Entities

Entity	Identity Attribute	Reason, why not a Value Object?
Rating	`ratingId : Long`	Has persistent identity (ratingId). Represents a unique user feedback record tied to one booking.

Value Objects

Value Object	Attributes	Why immutable / no identity?
Score	`value:Integer (1..5)`	Immutable. Validated on construction, throws if out of range.
Review	`vehicleScore:Score, providerScore:Score, comment:String`	Immutable grouping of both scores and optional comment. comment may be null.
RatingTarget	`vehicleId:Long, providerId:Long, bookingId:Long`	Immutable reference triplet. Identifies what and why is being rated.

Domain Services

Service	Key Operations
RatingSubmissionService	`submitRating(bookingId, userId, vehicleScore, providerScore, comment):Rating, validates booking completion and duplicate check`
RatingQueryService	`getVehicleRatings(vehicleId):List<Rating>, getProviderRatings(providerId):List<Rating>, getAverageScore(vehicleId):Double`

Repositories

Repository	Methods
RatingRepository	`findById(id), findByBookingId(bid), findByVehicleId(vid), findByProviderId(pid), existsByBookingId(bid):boolean, save(r)`

Domain Events

Domain Event	Trigger Condition
RatingSubmitted	RatingSubmissionService successfully creates and persists a new Rating

PlantUML Source:

```
@startuml BC-05: Rating
skinparam classBackgroundColor<<AggregateRoot>> #FFF3CD
skinparam classBackgroundColor<<ValueObject>> #D4EDDA
skinparam classBackgroundColor<<Service>> #CCE5FF
skinparam classBackgroundColor<<Repository>> #E2D9F3
skinparam classBackgroundColor<<DomainEvent>> #FFE0B2
skinparam classBorderThickness<<AggregateRoot>> 2
skinparam classBorderStyle<<ValueObject>> dashed

class Rating <<AggregateRoot>> {
  + ratingId : Long
  + userId : Long
  + createdAt : DateTime
}
class Review <<ValueObject>> {
  + vehicleScore : Score
  + providerScore : Score
  + comment : String
```

```
}
class Score <<ValueObject>> {
  + value : Integer
  --
  + {static} of(v) : Score
}
class RatingTarget <<ValueObject>> {
  + vehicleId : Long
  + providerId : Long
  + bookingId : Long
}
class RatingSubmissionService <<Service>> {
  + submitRating(bid,uid,vs,ps,c) : Rating
}
class RatingQueryService <<Service>> {
  + getVehicleRatings(vid) : List
  + getAverageScore(vid) : Double
}
interface RatingRepository <<Repository>> {
  + findByBookingId(bid) : Rating
  + findByVehicleId(vid) : List
  + existsByBookingId(bid) : boolean
  + save(r) : void
}
class RatingSubmitted <<DomainEvent>> {
  + ratingId : Long
  + bookingId : Long
  + vehicleScore : Integer
  + providerScore : Integer
}

Rating *-- Review
Rating *-- RatingTarget
Review *-- Score
RatingSubmissionService ..> RatingRepository
RatingQueryService ..> RatingRepository
RatingSubmissionService ..> RatingSubmitted : publishes
@enduml
```

Task 2: Event Storming

The team chose Event Storming to model the domain of the Instant Mobility Platform. Event Storming maps the system as a timeline of Domain Events (things that happened), with the Commands that trigger them, the Aggregates that process them, and the Policies that react to them.

Colour convention used in the Event Storming board:

- Orange , Domain Event (something significant happened, named in past tense)
- Blue , Command (an intention to change state, e.g. from a user or policy)
- Yellow , Aggregate (the DDD aggregate root that processes the command)
- Purple , Policy / Reaction (a rule: 'when event X happens, execute command Y')
- Green , Read Model (a view or query result used to support a decision)
- Tan , Actor (the person or system that issues a command)

2.1 Event Flow Summary

Phase	Actor	Command	Aggregate	Domain Event
Registration				
Registration	User	**Register User**	**UserAccount**	**UserRegistered**
Registration	Provider	**Register Provider**	**ProviderAccount**	**ProviderRegistered**
Registration	User	**Login**	**UserAccount**	**UserAuthenticated**
Vehicle Management				
Vehicle Management	Provider	**Create Vehicle**	**Vehicle**	**VehicleCreated**
Vehicle Management	Provider	**Update Vehicle**	**Vehicle**	**VehicleUpdated**
Vehicle Management	Provider	**Delete Vehicle**	**Vehicle**	**VehicleDeleted**
Search & Booking				
Search & Booking	User	**Search Vehicles**	**[Read Model]**	**VehiclesFound**
Search & Booking	User	**Book Vehicle**	**Booking**	**BookingCreated**
		Policy reaction:	**Mark Vehicle BOOKED**	
Search & Booking	—	**Update Status**	**Vehicle**	**StatusUpdated**
Ride & Completion				
Ride & Completion	User	**End Booking**	**Booking**	**BookingCompleted**
		Policy reaction:	**Compute Cost**	
Ride & Completion	—	**Compute Cost**	**Booking**	**CostComputed**
		Policy reaction:	**Trigger Payment**	
Ride & Completion	—	**Process Payment**	**Payment**	**PaymentSucceeded**
		Policy reaction:	**Release Vehicle**	

Ride & Completion	User	**Cancel Booking**	**Booking**	**BookingCancelled**
		Policy reaction:	**Release Vehicle**	
Rating				
Rating	—	**[Allow Rating]**	**[Policy]**	**RatingAllowed**
		Policy reaction:	**After BookingCompleted+PaymentSucceeded**	
Rating	User	**Submit Rating**	**Rating**	**RatingSubmitted**

2.2 Event Storming Narrative

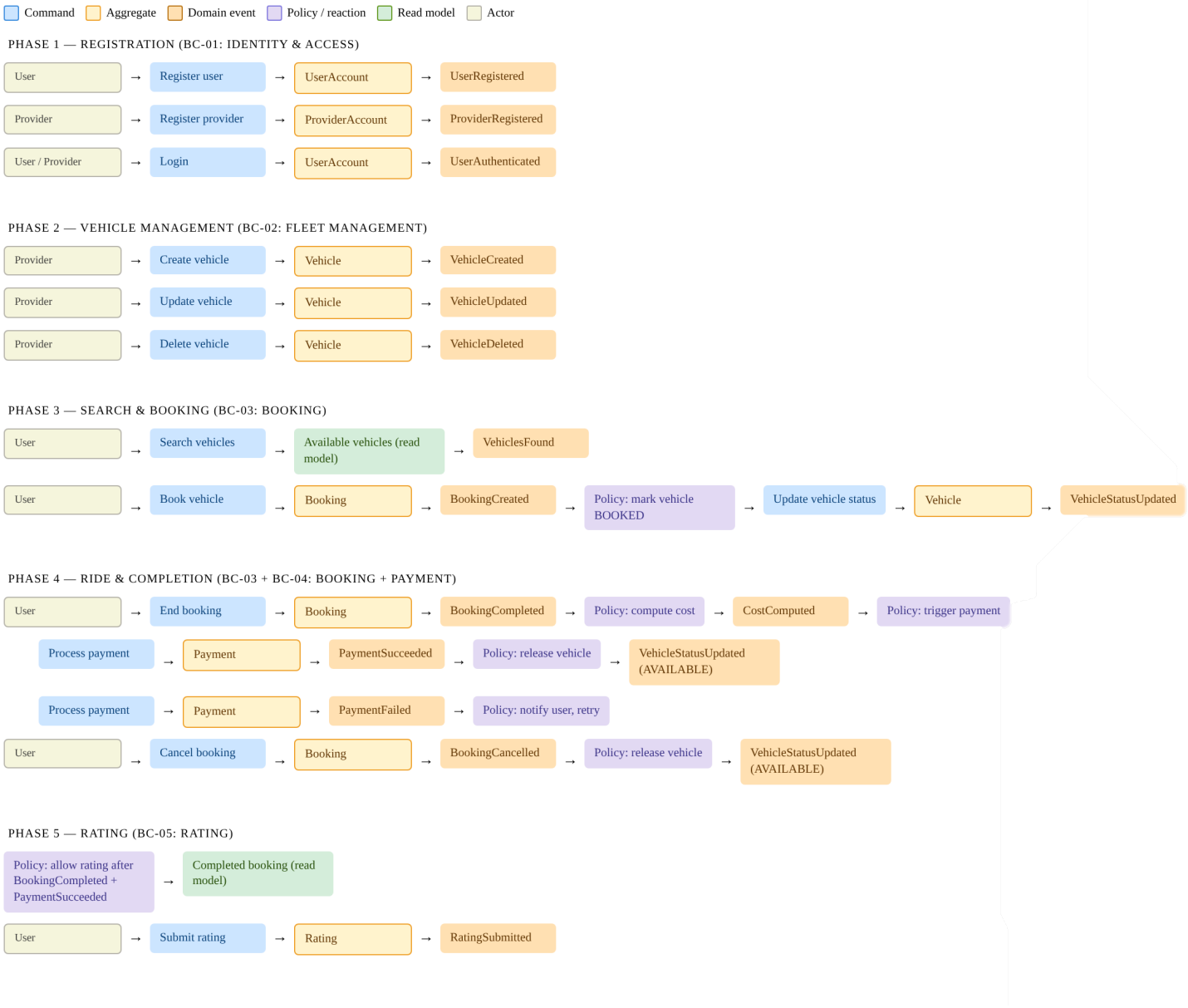
Phase 1, Registration: The process begins when a User or Provider registers an account (UserRegistered / ProviderRegistered). Login produces a UserAuthenticated event that provides the session token used in all subsequent operations.

Phase 2, Vehicle Management: Providers create, update, and delete vehicles in their fleet. VehicleCreated is the pivotal event, without it, no bookings are possible. VehicleStatusUpdated is raised both from this context (manual updates) and reactively from the Booking context.

Phase 3, Search & Booking: A User searches for available vehicles (producing a VehiclesFound read model). Booking a vehicle produces BookingCreated and triggers the policy 'Mark Vehicle BOOKED' which sends a status-update command back to Fleet Management.

Phase 4, Ride & Completion: Ending a booking triggers cost computation (CostComputed), which in turn triggers the PaymentTriggered event. The Payment context processes this asynchronously, PaymentSucceeded releases the vehicle (AVAILABLE again) via another policy. If the user cancels, BookingCancelled immediately releases the vehicle.

Phase 5, Rating: Only after both BookingCompleted and PaymentSucceeded have fired can the Rating context accept a rating submission. The policy guards this gate. RatingSubmitted is the final event in the happy path.


Command
Aggregate
Domain event
Policy / reaction
Read model
Actor
PHASE 1 — REGISTRATION (BC-01: IDENTITY & ACCESS)
User → Register user → UserAccount → UserRegistered
Provider → Register provider → ProviderAccount → ProviderRegistered
User / Provider → Login → UserAccount → UserAuthenticated
PHASE 2 — VEHICLE MANAGEMENT (BC-02: FLEET MANAGEMENT)
Provider → Create vehicle → Vehicle → VehicleCreated
Provider → Update vehicle → Vehicle → VehicleUpdated
Provider → Delete vehicle → Vehicle → VehicleDeleted
PHASE 3 — SEARCH & BOOKING (BC-03: BOOKING)
User → Search vehicles → Available vehicles (read model) → VehiclesFound
User → Book vehicle → Booking → BookingCreated → Policy: mark vehicle BOOKED → Update vehicle status → Vehicle → VehicleStatusUpdated
PHASE 4 — RIDE & COMPLETION (BC-03 + BC-04: BOOKING + PAYMENT)
User → End booking → Booking → BookingCompleted → Policy: compute cost → CostComputed → Policy: trigger payment
Process payment → Payment → PaymentSucceeded → Policy: release vehicle → VehicleStatusUpdated (AVAILABLE)
Process payment → Payment → PaymentFailed → Policy: notify user, retry
User → Cancel booking → Booking → BookingCancelled → Policy: release vehicle → VehicleStatusUpdated (AVAILABLE)
PHASE 5 — RATING (BC-05: RATING)
Policy: allow rating after BookingCompleted + PaymentSucceeded → Completed booking (read model)
User → Submit rating → Rating → RatingSubmitted